🧠 Introduction to Shell Scripting

🔹 What is a Shell?

A **shell** is a program that acts as an interface between you (user) and the operating system.

It takes your commands and tells the OS what to do.

👉 Examples of shells:

- Bash (most common)
- Sh (Bourne Shell)
- Zsh
- Ksh

🔹 What is Shell Scripting?

A **shell script** is a file containing a series of commands that are executed automatically.

Instead of typing commands one by one in the terminal, you write them in a file and run it.

📌 In simple terms:

> Shell scripting = Automating tasks using a file of commands

---

🔹 Why Use Shell Scripting?

Shell scripting helps in:

- ✅ Automating repetitive tasks
- ✅ Managing system operations
- ✅ Running backups
- ✅ Monitoring servers
- ✅ Deploying applications

🐚 Different Types of Shells in Linux/Unix

A **shell** is a command-line interpreter. There are multiple shells available, each designed with different capabilities.

🔹 1. Bourne Shell (sh)

- 📌 File: `/bin/sh`
- 👨‍💻 Developed by: Stephen Bourne
- 🟢 Oldest and most basic shell

Features:

- Supports basic scripting
- Lightweight and fast
- Used for system scripts

Limitation:

- No advanced features like arrays

🔹 2. Bourne Again Shell (bash)

- 📌 File: `/bin/bash`
- 🟢 Most commonly used shell (default in many Linux systems)

Features:

- Command history
- Tab completion
- Supports variables, loops, conditions
- Better scripting capabilities

Example:
echo "Hello from Bash"

👉 This is the shell you are using in most cases (especially in CentOS / Oracle VM).

🔹 3. Korn Shell (ksh)

- 📌 File: `/bin/ksh`
- 👨‍💻 Developed by: David Korn

Features:

- Combines features of sh and bash
- Better scripting than sh
- Supports associative arrays (in newer versions)

Use case:

- Enterprise environments (older systems)

🔹 4. C Shell (csh)

- 📌 File: `/bin/csh`
- Syntax similar to C programming language

Features:

- Good for interactive use
- History mechanism

Limitation:

- Not recommended for scripting (less reliable)

🔹 5. TENEX C Shell (tcsh)

- 📌 File: `/bin/tcsh`
- Improved version of csh

Features:

- Command auto-completion
- Better history handling
- User-friendly interface

🔹 6. Z Shell (zsh)

- 📌 File: `/bin/zsh`
- 🟢 Advanced and modern shell

Features:

- Powerful auto-completion
- Plugin support
- Themes (like Oh My Zsh)
- Better scripting than bash (in some cases)

Popular among:

- Developers
- DevOps engineers

🔹 7. Fish Shell (Friendly Interactive Shell)

- 📌 File: `/usr/bin/fish`
- 🟢 User-friendly shell

Features:

- Syntax highlighting
- Auto-suggestions
- No need to configure much

Limitation:

- Not fully compatible with bash scripts

📊 Comparison Table

Shell	Script Support	User Friendly	Popularity
sh	Basic	❌	Low
bash	Strong	✅	⭐⭐⭐⭐⭐
ksh	Strong	⚠️	Medium

csh	Weak	⚠️	Low
tcsh	Moderate	✅	Medium
zsh	Very Strong	⭐⭐⭐⭐⭐	High
fish	Moderate	⭐⭐⭐⭐⭐	Growing

🔍 How to Check Your Current Shell

echo $SHELL

🔄 How to List Available Shells

cat /etc/shells

🔄 How to Change Shell

chsh -s /bin/zsh

🔹 Basic Structure of a Shell Script

```
#!/bin/bash

# This is a comment
echo "Hello, World!"
```

Explanation:

- `#!/bin/bash` → Shebang (tells system to use Bash)
- `#` → Comment
- `echo` → Prints output

🔹 How to Create and Run a Script

Step 1: Create file
vi script.sh

Step 2: Add content
#!/bin/bash
echo "My first script"

Step 3: Give permission
chmod +x script.sh

Step 4: Run script
./script.sh

🧪 First Shell Script Example

🔹 **Script Code**

```
#!/bin/bash

echo "Hello, World!"
```

🔍 Explanation (Line by Line)

🔸 **1. `#!/bin/bash`**

- This is called a **shebang**
- It tells the system:
  👉 "Use the Bash shell to execute this script"

📌 Without this, the system may not know which shell to use.

🔸 **2. Blank Line**

- Improves readability
- Not mandatory, but good practice

🔸 **3. `echo "Hello, World!"`**

- `echo` is a command used to **print output** on the terminal
- `"Hello, World!"` is the message displayed

📌 **Output:**

Hello, World!

▶️ How to Run This Script (Step-by-Step)

Step 1: Create file

```
vi hello.sh
```

Step 2: Add code

```
#!/bin/bash
echo "Hello, World!"
```

Step 3: Give execute permission

```
chmod +x hello.sh
```

Step 4: Run script

```
./hello.sh
```

🎯 Expected Output

Hello, World!

💡 Important Learning Points

- First script always starts with **shebang**
- `echo` is the most basic command to display output
- Script must have **execute permission**
- `.sh` is a common naming convention

▶️ Ways to Execute a Shell Script

🔹 1. Execute Using Bash Command

```
bash script.sh
```

✅ **Explanation:**

- Directly tells the system to use **Bash interpreter**
- No need to give execute permission

📌 **Use Case:**

- Quick testing
- When script doesn't have execute permission

🔹 2. Execute Using sh Command

sh script.sh

✅ **Explanation:**

- Uses **Bourne shell (sh)** to run script
- Works even without execute permission

⚠️ **Important:**

- Some Bash features may not work with sh

🔹 3. Execute as an Executable File (./)

./script.sh

✅ **Requirements:**

- Script must have execute permission:

chmod +x script.sh

- Must include shebang:

#!/bin/bash

📌 **Explanation:**

- Runs script like a program

🔹 4. Execute with Absolute Path

/home/user/script.sh

✅ **Explanation:**

- Runs script using full path
- Useful when not in the same directory

🔹 5. Execute with Relative Path

../script.sh

✅ **Explanation:**

- Runs script from another directory using relative location

🔹 6. Execute by Adding to PATH

Step 1: Move script
mv script.sh /usr/local/bin/

Step 2: Run directly
script.sh

✅ **Explanation:**

- No need to type `./`
- Works like a system command

🔹 7. Execute Using `source` Command

source script.sh

OR

. script.sh

✅ **Explanation:**

- Runs script in **current shell**
- Does NOT create a new shell

📌 **Use Case:**

- When you want variables to remain available after execution

---

📊 Quick Comparison

Method	Permission Needed	New Shell	Use Case
bash script.sh	❌	✅	Testing
sh script.sh	❌	✅	Basic scripts
./script.sh	✅	✅	Normal execution
source script.sh	❌	❌	Env variables
PATH method	✅	✅	Production

🎯 Important Interview Point

👉 Difference:

- `./script.sh` → Runs in **new shell**
- `source script.sh` → Runs in **same shell**

🧠 Simple Analogy

- `bash script.sh` → "Run this using Bash"
- `./script.sh` → "Execute like a program"
- `source script.sh` → "Run inside current environment"

🧠 Concept: New Shell vs Current Shell

🔹 New Shell (Child Process)

When you run:

bash script.sh

👉 A **new shell (child process)** is created
👉 Script runs inside it
👉 After execution → shell is destroyed

📌 Result:

- Changes stay inside script only ❌
- Nothing affects your current terminal

🔹 Current Shell (Same Process)

When you run:

source script.sh

OR

. script.sh

👉 Script runs in the **current shell itself**
👉 No new shell is created

📌 Result:

- Changes remain in your terminal ✅
- Environment gets modified

📊 2. Key Differences

Feature	New Shell (`bash script.sh`)	Current Shell (`source script.sh`)
Process	New child shell	Same shell
Variable persistence	❌ Not retained	✅ Retained

Environment changes	❌ Lost	✅ Persist
Safety	✅ Safe (isolated)	⚠️ Can affect system
Use case	Running scripts	Setting environment

🧪 3. Simple Practical Example

🔹 **Script: `env.sh`**

```
#!/bin/bash

name="DevOps"
export PROJECT="Shell"
echo "Inside script: $name $PROJECT"
```

▶️ Case 1: New Shell

```
bash env.sh
echo $name
echo $PROJECT
```

✅ **Output:**

```
Inside script: DevOps Shell
(empty)
(empty)
```

👉 Variables are lost ❌

▶️ Case 2: Current Shell

```
source env.sh
echo $name
echo $PROJECT
```

✅ **Output:**

```
Inside script: DevOps Shell
DevOps
Shell
```

👉 Variables are available ✅

🌍 4. Real-World Examples

Environment Setup (Most Common)

File: `env.sh`
export DB_HOST=localhost
export DB_USER=admin
export APP_ENV=dev

Run:
source env.sh

👉 Now your application can use:

echo $DB_HOST

✅ This is used in:

- DevOps
- Application setup
- Docker/Kubernetes environments

⚠️ 5. When to Use What?

✅ Use `bash script.sh` when:

- Running automation scripts
- Backups, cleanup, deployments
- You don't want side effects

✅ Use `source script.sh` when:

- Setting environment variables
- Updating PATH
- Loading configuration
- Working with `.bashrc`

💬 Comments in Shell Scripting

🔹 What is a Comment?

A **comment** is a line in a script that is **ignored by the shell**.

👉 It is used to:

- Explain code
- Improve readability
- Help others understand the script

🔹 How to Write Comments

🔹 Single-Line Comment

```
# This is a comment
echo "Hello World"
```

👉 Anything after # is ignored

🔹 Example

```
#!/bin/bash

# This script prints a message
echo "Welcome to Shell Scripting"
```

🔹 Inline Comments

You can also write comments on the same line:

```
echo "Hello"   # This prints Hello
```

🔹 Multi-Line Comments (Important)

❗ Shell scripting does NOT have a direct multi-line comment like other languages.

✅ **Method 1: Use multiple #**

```
# This is line 1
# This is line 2
# This is line 3
```

✅ **Method 2: Using : << (Block Comment Trick)**

```
: << 'COMMENT'
This is a multi-line comment
It will not be executed
COMMENT
```

👉 Explanation:

- : is a no-op command (does nothing)
- Everything inside block is ignored

🔹 Shebang is NOT a Comment

```
#!/bin/bash
```

👉 Looks like a comment, but it is **not**

- It tells the system which shell to use
- Must be the first line in script

🔹 Best Practices

- ✔ Use comments to explain logic
- ✔ Keep comments short and meaningful
- ✔ Avoid obvious comments

❌ Bad:

```
echo "Hello"  # print hello
```

✅ Good:

```
# Display welcome message to user
```

```
echo "Hello"
```

🔹 When to Use Comments

- Explaining complex logic
- Mentioning author/date
- Describing purpose of script
- Temporarily disabling code

🚀 Summary

- **sh** → Basic, old
- **bash** → Default, widely used
- **ksh** → Enterprise scripting
- **csh/tcsh** → C-like syntax
- **zsh** → Modern & powerful
- **fish** → Beginner-friendly